1. The repository and its purpose

CMPE 249 Detector Evaluation Repository: overview, workflow and project use. The cmpe249-detector-evaluation repository packages our Track A object-detection study as an organized, repeatable software workflow. It brings together experiment configuration, dataset checks, GPU inference, official metric computation, efficiency measurements, analysis, tests and report artifacts. Its main purpose is to make each reported result traceable to an input set, model configuration and saved output.

For our project, we compared the supplied COCO-pretrained YOLOv8n, YOLO11n and RT-DETR-L checkpoints across COCO and a driving domain. The first two are YOLO configurations; RT-DETR provides a second detector family [2,3,5,6,9]. We performed inference and evaluation without training or fine-tuning. We asked how these checkpoint configurations differ in accuracy, prediction cost and behavior across datasets and difficult local scenes.

The current repository is the cleaned and modularized form of the workflow used for those experiments. The full experiment was completed before the source was reorganized. Its compact tables and evidence were then copied into results/published/, while large original predictions and datasets remained in the parent project. We subsequently tested the relocated package independently. The published measurements and the refactoring checks therefore document two different stages of the project.

The repository is prepared for GitHub distribution, with a single entry point, a src-layout Python package, readable configuration, tests and documentation. The report describes script responsibilities but includes no source listings. Code availability: the author will add the GitHub repository URL and exact commit or release identifier. No URL or public release is assumed here.

Repository capability	Use in the project
Controlled evaluation	Same primary input settings and saved image manifests across models
Official metrics	COCOeval on COCO and converted KITTI; saved predictions for reevaluation
Efficiency study	Separate synchronized timing and memory measurements
Verification images	Local scene comparisons with explicit limits when labels are absent
Evidence and handoff	Tables, figures, reports, tests, logs and verified result archives

2. Organization and architecture

The public entry point is run_pipeline.py. It locates the repository's src directory and delegates to cmpe249_eval.cli. The optional installed command cmpe249-eval exposes the same interface. Users do not need to edit internal imports or execute individual module files to run a stage. Package paths are resolved relative to the repository, so invoking the entry point from another working directory does not redirect its data or outputs.

The package separates responsibilities. Configuration and basic I/O support dataset and geometry utilities; evaluation uses those utilities and official COCOeval. The pipeline plans cases and asks the runtime to launch isolated workers. Analysis consumes completed artifacts, and the artifact/report modules produce the handoff. A separate verification path supports an ordinary image folder without requiring COCO or KITTI to be downloaded.

Committed defaults live in configs/default.json. An ignored configs/local.json can override checkpoint, dataset and reporting-interpreter locations without putting personal machine paths into Git. On the original machine, those overrides reuse the existing dtect environment's compatible inputs and the parent checkpoints. New outputs go under outputs/<run-id>/; published original results never act as an inference cache.

Location	What belongs there
src/cmpe249_eval/	Reusable configuration, data, metric, runtime, analysis and reporting modules
configs/	Portable protocol defaults, local-configuration example and ignored local overrides
tests/	CPU evaluator, path-relocation, image-inventory and worker-contract tests
data/verification/images/	The user's verification/benchmark source images; eligible for Git tracking
data/datasets/ and weights/	Large validation datasets and checkpoints; ignored by Git
outputs/	New run directories, manifests, predictions, overlays and logs; ignored by Git
results/published/	Compact evidence from original run 20261103_track_a
reports/ and docs/	PDF reports, architecture, migration, reproduction and image-upload guidance
.github/workflows/	Configured CPU checks for future GitHub pushes and pull requests

3. How the workflow operates

A run starts by resolving configuration and assigning a run ID. The doctor command inventories dependencies, CUDA and input paths. The test command runs the evaluator/data regression checks without loading a detector. For the full experiment, prepare builds the dataset audit and fixed manifests; smoke then adds small GPU checks before expensive primary inference begins.

The pipeline writes the planned matrix before running cases. Each model/configuration is passed as a job file to a fresh worker subprocess. The worker loads the correct YOLO or RT-DETR predictor, checks tensor properties and either exports original-coordinate predictions or performs repeated timing. Only one owned model worker runs at a time. A missing checkpoint is an explicit error rather than an automatic download or CPU substitution.

Prediction JSON is written before metric computation. This separates expensive inference from evaluation: the evaluate command can consume a ground-truth file and an existing prediction file without another model pass. Analysis reads the saved metrics and samples to create tables, plots and comparisons. Packaging then checks the generated archives and separates bulky prediction evidence from the smaller report-oriented export.

Compatible completed inference jobs can be reused when a run ID is resumed. Signatures include the configuration, image metadata and content hashes, category definitions, checkpoint hash and package code hashes. Changing a signature invalidates the previous completion marker. An OOM is retried once in a clean process with identical settings; other failures are logged. These signatures do not independently guarantee that a driver, operating system or every external dependency is unchanged.

Stage or command	Main input -> persisted output
prepare	Dataset configuration -> audited GT, image lists, hashes and GT overlays
smoke	Evaluator fixtures and a few KITTI images -> assertions and GPU tensor evidence
coco / kitti	Fixed labeled image manifests -> prediction JSON and official metric exports
benchmark	Decoded verification images -> repeated latency samples and memory statistics
ablation	Fixed 200-image KITTI subset -> resolution/precision accuracy comparisons
analyze	Saved outputs -> CSV/LaTeX tables, figures and interpretation
package	Report and evidence artifacts -> integrity-checked ZIP exports
verify-images	User image folder -> predictions, overlays, manifest and output summary

4. Methods encoded in the repository

The primary protocol uses GPU device 0, square input 512, batch one and FP32, with confidence 0.001 and an export cap of 300 detections per image. YOLO NMS IoU is 0.70; RT-DETR is NMS-free. The worker checks actual model/input dtype, CUDA placement, batch one, evaluation mode and disabled gradients. Both paths use RGB intensities normalized by 255, but YOLO preserves aspect ratio with padding and RT-DETR stretches to the square. Equal nominal input size therefore does not mean identical preprocessing.

COCO evaluation uses all 5,000 val2017 images and 36,781 instance annotations across 80 categories [1]. KITTI uses a fixed sample of 500 images from the existing prepared 1,496-image validation split [4]. The parent list is sorted, sampled using random.Random(42), and saved for every model to reuse. A separate fixed sample of 200 of these images supports the labeled ablation. Image IDs, file dimensions, category references, finite boxes, positive extents and missing-versus-empty labels are checked.

KITTI's normalized centers and sizes are converted to original-pixel corners, clipped to image bounds and exported as COCO xywh boxes. Detector class names map to the actual category IDs instead of assuming contiguous IDs are interchangeable. Person/pedestrian, car and truck form the primary shared vocabulary. Bicycle/cyclist is supplementary because object definitions and box extents differ. Van, sitting-person, tram and misc annotations are outside the scored mapping.

Official pycocotools.COCOeval provides bbox AP50-95, AP50, AP75, size AP, per-class AP and AR1/10/100 [7]. It uses ten IoU thresholds from .50 to .95, 101 recall samples and maxDets [1,10,100]. Undefined metrics remain unavailable rather than zero. COCO's original area/crowd metadata is preserved, whereas converted KITTI uses box area. Prepared KITTI lacks DontCare, truncation, occlusion and difficulty fields; its reported values are converted COCO-style scores, not official KITTI benchmark results.

Shared concept	COCO GT count	Selected KITTI GT count	Role
person / pedestrian	11,004	296	Primary
car	1,932	1,860	Primary
truck	415	79	Primary
bicycle / cyclist	316	114	Approximate supplementary

5. How we used the workflow for our project

The completed original run, 20261103_track_a, used six primary evaluations: three models on COCO and the same three on the fixed KITTI subset. It then added 20 distinct timing cases and 12 labeled ablation cases, giving 38 planned configurations. The three 512/FP32 ablation baselines were obtained by filtering compatible primary KITTI predictions to the 200-image subset; the remaining ablation settings required new inference.

The timing study changed one factor at a time around 512/FP32/confidence .25: resolution 384/512/640, FP32 versus FP16, confidence .10/.25/.50, and YOLO NMS .50/.70. Duplicate baselines were removed and RT-DETR had no NMS sweep. All 19 local images were decoded outside measurement. Each timing case used ten explicit warmups and five repetitions per image, for 95 measured calls per configuration and 1,900 across the 20 cases.

A monotonic clock and CUDA synchronization bounded the full prediction call. This includes preprocessing, transfer, forward, postprocessing, coordinate restoration and API overhead, while excluding image decoding, export and drawing. Framework forward timing is retained separately. FPS is 1000 divided by mean latency in milliseconds. Peak PyTorch allocated and reserved memory is recorded after warmup; it is not total device VRAM use.

The measured platform was the existing Python 3.13.6 dtect environment, PyTorch 2.14.1+cu130, Ultralytics 8.4.198 and an NVIDIA RTX 5070 Ti Laptop GPU. The recorded Windows power scheme was Silent, with other desktop GPU activity present. Timing variation was retained rather than deleting slow calls. These are checkpoint-level results on that setup, not universal architecture rankings.

The results support two project analyses. RT-DETR-L had higher COCO AP than YOLOv8n, at about 2.58 times its measured full-prediction latency. Shared macro AP decreased on KITTI for all three models, but car AP increased while person and truck AP declined. This class-level distinction prevents treating a macro drop as uniform transfer failure. Differences remain confounded by model capacity, annotation conventions, scene composition and preprocessing.

Model	COCO 80-class AP	Shared COCO AP	Shared KITTI AP	Full latency ms
yolov8n.pt	0.3518	0.3567	0.1943	10.59
yolo11n.pt	0.3638	0.3632	0.1969	14.10
rtdetr-l.pt	0.5082	0.5017	0.2860	27.27

Original experiment component	Completed configurations
COCO + KITTI primary	6
One-factor timing study	20
Fixed-subset labeled ablation	12 (including 3 reused prediction subsets)
Total	38 successful; no final experiment OOM or inference failure

6. Verification images and the local scene study

The original project also used 19 local photographs to study application timing and qualitative behavior. Three untimed visual prediction passes produced 57 image/model records and one comparison panel for each image. The separate 22-page benchmark report combines the original with all three model outputs, per-image timing summaries, class counts and inspected scene descriptions. It reused the original timing samples instead of timing image writing.

The repository's current upload location is data/verification/images/. At the time this report was revised, all 19 of the 19 uploaded image files matched the original benchmark SHA-256 hashes, with 0 mismatches. The folder also contains a Python image utility; the loader selects supported image formats and does not execute that file. This file match establishes input identity, not a new inference result.

The current verify-images command can process any number of supported images in that directory, including subfolders. It reads, decodes and hashes source files, assigns manifest IDs and launches one detector worker at a time. It writes per-model prediction JSON, original-coordinate overlays and a CSV summary under outputs/<run-id>/verification/. It uses confidence .25 for display. That is separate from the primary accuracy protocol's .001 confidence floor.

If real labels are available, data/verification/annotations.json can supply COCO-format image records, categories and boxes. Every input image must be uniquely represented, dimensions must match and class names must be recognized. The command preserves supplied IDs and can evaluate those labels at the display threshold. Without labels it generates qualitative evidence only: AP, precision, recall and false-positive rates cannot be established.

For example, traffic.jpg produced 22 outputs from YOLOv8n, 21 from YOLO11n and 140 from RT-DETR-L in the original .25 visual pass. This is a documented output-count difference, not evidence that all extra boxes are correct. Likewise, visible crowding or blur suggests a useful case for inspection but does not supply an error rate without labels. The original 19-image measurements have not been represented as a new complete run of the reorganized package.

Artifact or location	Meaning
data/verification/images/	Source images supplied by the user; eligible for Git tracking
data/verification/annotations.json	Optional real ground truth; not automatically invented
outputs/<run-id>/verification/	New image manifest, per-image summary and overlays
outputs/<run-id>/experiments/	Per-model jobs, predictions, tensor evidence and worker logs
reports/benchmark_19_report.pdf	Historical 22-page report covering the original 19 images

7. Main modules and validation of the repository

The refactoring kept the scientific mechanisms and moved them into explicit modules. run_pipeline.py is now the authoritative entry point; the old prototype entry points remain outside this repository. The table maps the files a reader will encounter on GitHub to their roles, without reproducing their source code.

We checked the reorganized package with eleven repository/unit tests; one of those runs the fifteen official-evaluator/data regression cases. Coverage includes path resolution from an unrelated working directory, local configuration merging, image inventory, real annotation IDs and the subprocess package contract. The planned matrix still has 38 unique cases and no RT-DETR NMS sweep.

An additional local integration check used the existing bus image from an ignored output-input folder. All three relocated workers completed on CUDA, verified FP32/batch-one/evaluation-state conditions and saved overlays. Output counts were five for each YOLO model and ten for RT-DETR-L. These counts are not accuracy results, and the test did not rerun full COCO or KITTI. The GitHub Actions CPU workflow is configured but has not been claimed to have executed remotely.

Original-run verification is retained separately: fifteen evaluator checks, 41 artifact-integrity checks and exact reevaluation of a saved shared KITTI prediction export. Keeping those results separate from repository checks allows a reader to distinguish measured project findings from evidence that the reorganized software still operates correctly.

File(s) under src/cmpe249_eval/	Responsibility
cli.py / config.py	Public commands, common settings and portable/local path resolution
io.py / geometry.py / constants.py	Atomic exports, hashes, image discovery and validated box conversion
datasets.py / evaluation.py	Dataset audits/manifests and official COCOeval metrics/arrays
pipeline.py / runtime.py / worker.py	Case planning, isolated execution, compatible caching and GPU checks
verification.py	User images, optional labels and separately generated overlays
validation.py / integrity.py	Asserted regression cases and saved-output reconciliation
analysis.py	Tables, plots, class/domain comparisons and labeled qualitative diagnostics
report_content.py / render_report.py / artifacts.py	Generated narratives, optional PDF rendering and verified result packages
tests/test_repository.py	Repository tests; located in tests/, outside the runtime package

8. Reproducibility, project value and limitations

To reuse the project, a reader configures the three checkpoints and validation data, inspects doctor output, runs the CPU tests and then chooses verify-images or the full staged workflow. Existing CUDA installations can be reused; optional reporting dependencies may use a separate interpreter. The committed configuration is portable, while local overrides, large datasets, weights and new outputs are ignored by Git. Verification images can be tracked intentionally.

The repository's value is the chain from configuration and audited image lists to saved predictions, official metrics and report-ready evidence. We used that chain to support speed-accuracy analysis, shared-class domain comparison, resolution/precision ablations and qualitative inspection. Fixed subsets and original prediction exports make disagreements inspectable and metrics repeatable, rather than relying only on a final spreadsheet or framework console output.

Limits remain explicit: model capacities and resizing differ; prepared KITTI lacks official ignore/difficulty metadata; bicycle/cyclist is approximate; object-area conventions differ; laptop timings vary; no AP confidence interval or cross-device determinism guarantee was established. The 19 unlabeled images support output/timing observations, not measured accuracy. The full original experiments were not rerun merely to reorganize the source.

Code availability and AI-use statement. The author will provide the GitHub URL and pinned commit/release. An AI coding agent implemented and executed the local workflow, inspected artifacts and produced these documents from saved results. The student is not represented as having independently verified every source or result. This report contains explanations and filenames only; script source belongs in the repository. References below were checked during the original experiment, with version-specific behavior grounded in the installed implementation.

[1] Lin et al. Microsoft COCO: Common Objects in Context. ECCV 2014. https://arxiv.org/abs/1405.0312

[2] Carion et al. End-to-End Object Detection with Transformers. ECCV 2020. https://arxiv.org/abs/2005.12872

[3] Zhao et al. DETRs Beat YOLOs on Real-time Object Detection. CVPR 2024. https://arxiv.org/abs/2304.08069

[4] Geiger, Lenz and Urtasun. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. CVPR 2012. https://www.cvlibs.net/publications/Geiger2012CVPR.pdf

[5] Ultralytics. YOLOv8 model documentation. https://docs.ultralytics.com/models/yolov8/

[6] Ultralytics. YOLO11 model documentation. https://docs.ultralytics.com/models/yolo11/

[7] COCO Consortium. Official Python COCOeval implementation. https://github.com/cocodataset/cocoapi/blob/master/PythonAPI/pycocotools/cocoeval.py

[8] Ultralytics. Official framework source repository. https://github.com/ultralytics/ultralytics

[9] Ultralytics. RT-DETR model documentation. https://docs.ultralytics.com/models/rtdetr/